26 最大流

Part VI Graph Algorithms

本章定位

從可行流與割的上下界出發，說明增廣路為何需要撤回舊決策，再連到配對、節點容量與push–relabel。數例同時核對流量守恆、殘餘容量及最優性證書。

前置知識

有向圖、BFS與求和記號；本章會建立流量守恆、殘餘網路及割的定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 flow network 與 flow value。
- 用 residual graph 修正 Ford-Fulkerson 的增廣。
- 推導 max-flow min-cut 並連接 bipartite matching。
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

容量網路有有限頂點V、不同的source s與sink t、容量c(u,v)≥0。無self-loop，非邊容量0；平行同向邊可合併容量。記n=|V|≥2、m為正容量有向邊數；允許孤點與不在s-t路上的點。

本章統一採反對稱淨流f(u,v)=−f(v,u)，並要求f(u,v)≤c(u,v)，所以完整範圍是−c(v,u)≤f(u,v)≤c(u,v)。內點u≠s,t滿足Σ_v f(u,v)=0；流值|f|=Σ_v f(s,v)，是source淨流出，不是邊流絕對值的和。

不能再對全部有序對要求f≥0，否則反對稱會迫使f全為0。容量可為實數；時間先按精確算術操作數計，整數／有理數的位元成本另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非負 Arc Flow 與淨流不能混算

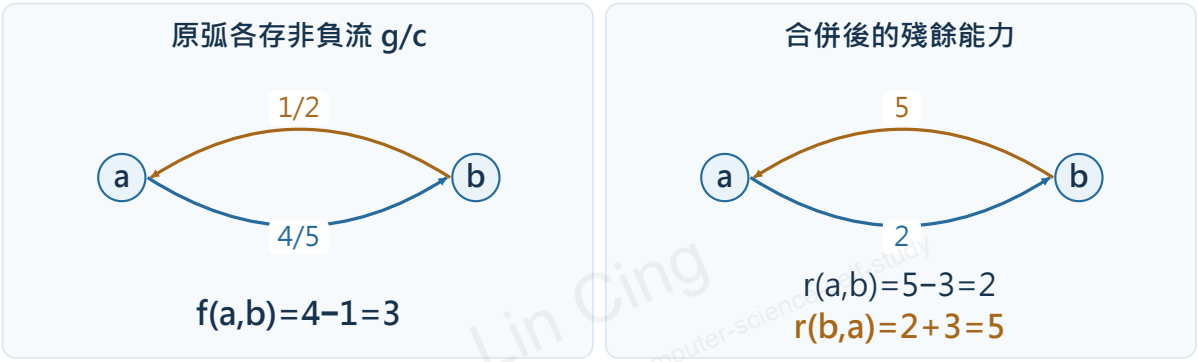
另一表示對原邊存0≤g(u,v)≤c(u,v)，不要求g反對稱。令f(u,v)=g(u,v)−g(v,u)，守恆與淨值不變；雙向都有流時，可取消min(g(u,v),g(v,u))而不改任何點的淨收支。

反過來，給淨流f，令g(u,v)=max(f(u,v),0)。容量上界使g≤c，且g(u,v)−g(v,u)=f(u,v)，故可恢復非負arc flow，兩模型的最大淨值相同。

例c(a,b)=5、c(b,a)=2，若g(a,b)=4、g(b,a)=1，淨f(a,b)=3、f(b,a)=−3。取消雙向各1後，原arc流成3與0。淨流的正向殘餘5−3=2，反向殘餘2−(−3)=5，包含撤回3與新增反向2的可能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反向容量與撤回舊流，各自貢獻多少？



反向5 = 原本可反向送2 + 可撤回淨流3

左圖只看一對反平行弧：c(a,b)=5、c(b,a)=2，原弧流為4與1，淨流為3。右圖b→a可增加5：先撤回淨流3，還能使用原本反向容量2。**殘餘弧不等於原圖有同方向的邊；也不能把反向殘餘一律當作只有撤流。**

Cut 的上界來自內部相消

Cut(S,T)分割全部V，s∈S、t∈T；c(S,T)=Σ_(u∈S,v∈T)c(u,v)，只算S→T方向。把S內所有點的淨流出相加，source貢獻|f|，其他內點守恆為0；S內每對f(u,v)+f(v,u)=0相消，所以

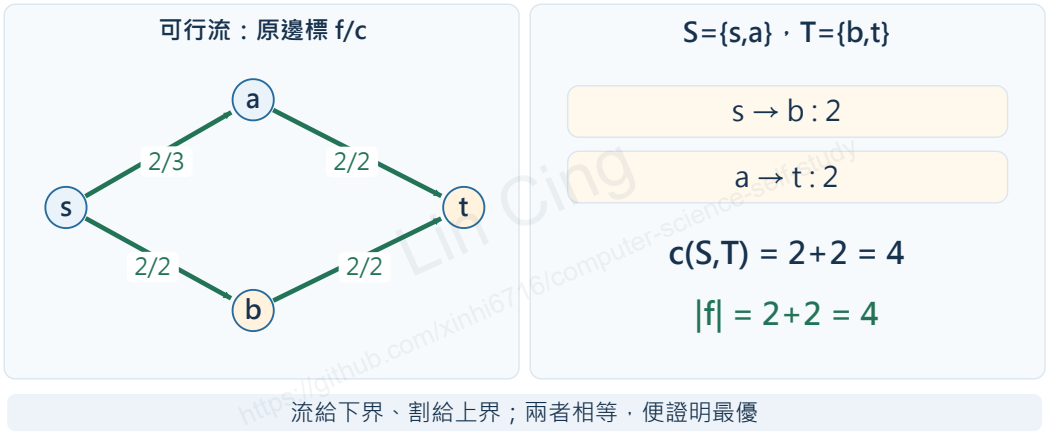
$$|f| = \sum_{u \in S, v \in T} f(u,v) \le \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

跨割必算淨值，不是只加正流。任何可行流給最大值下界，任何cut給上界；兩者相等便是可核驗的最佳性證書。

同一網路交付流與割兩份證書

容量s→a=3、a→t=2、s→b=2、b→t=2。各沿s→a→t、s→b→t送2，內點守恆且不超容量，流值4。

取S={s,a}、T={b,t}，跨割容量為s→b的2加a→t的2，總4。因此最大流同時≥4且≤4，答案就是4。



流給下界、割給上界；兩者相等，便證明最優

只給source總出容量或sink總入容量不能推定中間可達性；此例給了全部邊、可行流與等值cut，才足以證最優。

多 Source／Sink 與有限超邊

若供給端A、需求端B各無個別上限，且A∩B=∅，加supersource s'→各a∈A、各b∈B→supersink t'。超邊容量可用原邊容量總和C，不必實作∞；任何有效運輸量都不超過全部原容量和。若端點有各自上限，就用其上限，不能默認無限。

把原供需淨量接到超邊即得新守恆流；移去超邊則恢復原供需量，總值不變。這是兩向對應，不只是畫一個新source。

若使用非負arc表示且想消除antiparallel原邊，可將u→v容量c改成u→x→v、兩段各容量c。新點x守恆迫使兩段同流，與原邊一一對應。容量不能各改成c/2。

26.2 The Ford-Fulkerson method

殘餘容量$c_f(u,v)=c(u,v)-f(u,v)\geq 0$，residual graph只保留$c_f>0$的有向邊；鄰居候選必含原邊的反向，即使反向原容量0。

對residual簡單s-t路p，取$\delta=\min_{(uv\in p)}c_f(u,v)>0$。沿每條uv同時加$f(u,v)+=\delta$、$f(v,u)-=\delta$。正向不超容量，反向下降也不超其上界；反向下界則等價於正向上界。路內每點的入／出改變抵消，source淨流出增δ，故新f仍可行且值嚴格增加δ。

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u,v) for (u,v) in p)
    for (u,v) in p:
        f(u,v)+=delta; f(v,u)-=delta
return f
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣

原邊s→a、s→b、a→b、a→t、b→t容量皆1：

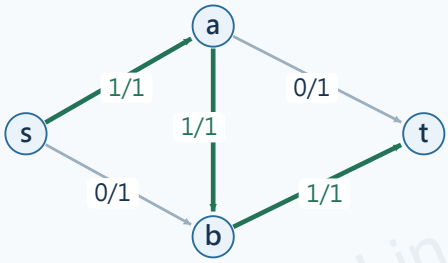
階段	路徑、δ	(f_sa,f_sb,f_ab,f_at,f_bt)	值
初始	—	0, 0, 0, 0, 0	0
第1次	s→a→b→t，1	1, 0, 1, 0, 1	1
第2次	s→b→a→t，1	1, 1, 0, 1, 1	2

第二路的b→a取消舊a→b流，使它由1回0。最後是s→a→t與s→b→t各1。若只保留原方向，第一輪後會錯把「沒有原方向路」當成「沒有residual路」，誤答1。

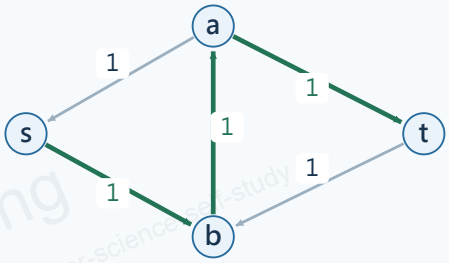
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣 （續）

第1輪後：原邊的 f/c



同一狀態：殘餘容量 r

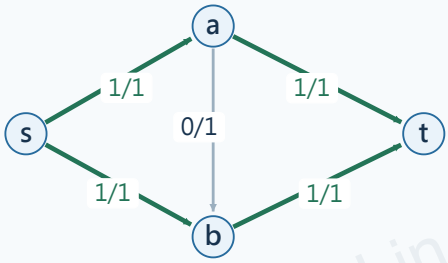


右圖綠路 s→b→a→t 的瓶頸為1；b→a 是撤回能力

若直接儲存殘餘邊，等價更新是r(u,v)−=δ、r(v,u)+=δ；只減正向容量會丟失撤回能力。

撤流後，逐條核對原邊與最優值

第2輪後：可行流值2



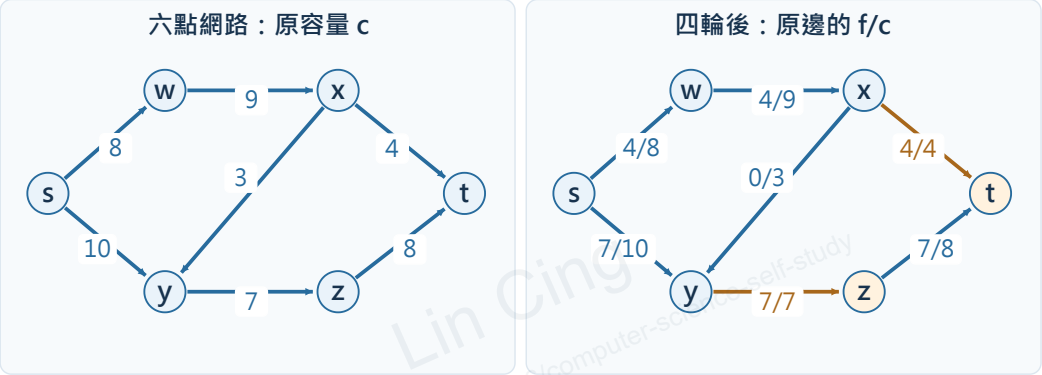
逐條原邊核對

s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b由1退到0；source兩條出邊已飽和，cut {s} 容量2

第二次增廣只更新s→b、b→a、a→t。b→a使淨f(a,b)減1，不是把a→b的原容量刪掉。最後a、b各收1送1；流值2等於割{s}容量2，故已最優。剩下的殘餘a→b仍有容量1，但source到不了a，不能用它再增廣。

六點網路：同一個最優值，可以怎樣走到？



S={s,w,x,y}：跨割容量 x→t:4 加 y→z:7，等於流值11

原圖只有x→y:3，沒有y→x。右圖是四輪增廣後的原邊流量f/c，並非殘餘容量。下面固定流量向量順序為(sw,wx,xy,xt,sy,yz,zt)，逐輪核對中間的撤流。

前兩輪：先走x→y，再沿y→x撤回

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
初始	—	—	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t；(8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t；(10,3,4)	3	(3,3,0,3,3,3,3)

第1輪才創造出殘餘y→x:3。第2輪沿它送3，使原f(x,y)由3變0；不是在原圖額外送一條y→x流。此時x的收支是wx:3流入、xt:3流出；y的收支是sy:3流入、yz:3流出。source淨流出3+3=6，兩輪都增加3。

後兩輪：用流量守恆與割完成檢核

向量仍依(sw,wx,xy,xt,sy,yz,zt)排列。

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
3	s-y-z-t；(7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t；(5,6,1)	1	(4,4,0,4,7,7,7)

流值6→10→11。最終w收4送4，x收4送4，y收7送7，z收7送7。殘餘從s可達S={s,w,x,y}，到不了z或t；原圖跨割只有x→t:4與y→z:7，合計11。**可行流11給下界，割11給上界，所以最大流恰為11。** 只看source原出容量8+10=18，無法得到這個答案。

Max-flow Min-cut 的三向等價

對可行f，以下等價：(i)f最大；(ii)residual無s-t路；(iii)存在cut使|f|=c(S,T)。

(i)⇒(ii)：若有路，增廣得到更大可行流，矛盾。

(ii)⇒(iii)：令S為residual從s可達點、T=V\S。t不可達，所以為合法cut。對u∈S,v∈T，c_f(u,v)=0，否則v也可達；故f(u,v)=c(u,v)，由cut淨流等式得|f|=c(S,T)。

(iii)⇒(i)：任一可行g都有|g|≤c(S,T)=|f|，所以f最大。

定理不要求整數容量，也不要求特定選路。整數性用於終止與成本，不是這個等價定理的前提。

整數性與偽多項式成本

整數容量從f=0開始，residual與δ皆整數，更新後仍整數。每次值至少增1，且有有限cut上界，所以必終止，並得到逐邊整數的最大淨流。設最優值F*，增廣至多F*次。

以adjacency list、一次O(n+m)配置及visited時間戳記，找路只初始化本輪觸及的點；每個新見非source點對應一條掃過的邊，每輪O(m+1)。總O(n+m+(m+1)F*)。F*可能對binary輸入長度指數大，因此是pseudopolynomial。

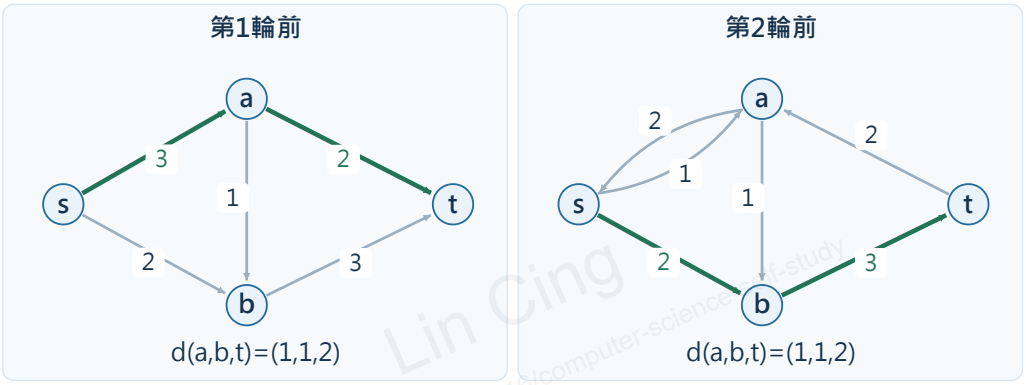
有理容量同乘分母乘積D可整數化；bitlen(D)不超各分母位元數總和，卻不代表D的數值小，所以仍可能有巨大增廣次數。無理容量下「每次至少1」不成立，不能由此證明宣稱任意選路終止。以下BFS版本改用容量值無關的輪數界。

Edmonds–Karp：殘餘邊數最短，不是容量最小

每輪以BFS找最少邊數s-t路，parent回溯增廣。d_f(v)為從s到v的最短邊數，不可達為∞。容量s→a=3、s→b=2、a→b=1、a→t=2、b→t=3，鄰居a先於b：

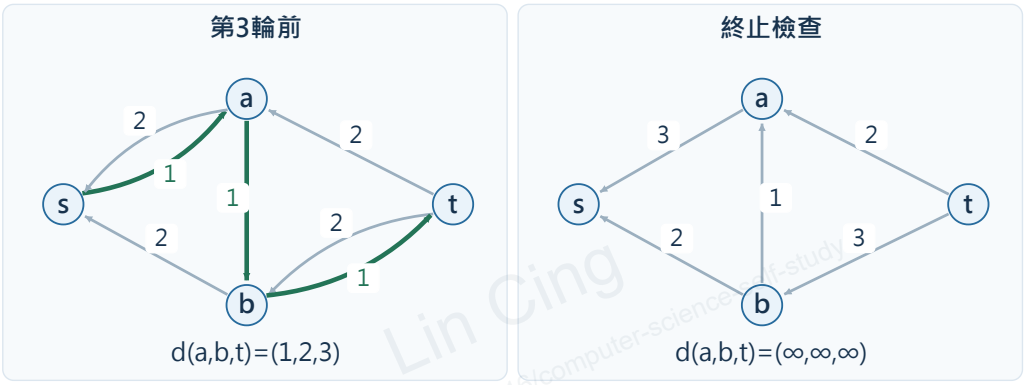
增廣前	d(a),d(b),d(t)	路徑與δ	增廣後f_sa,f_sb,f_ab,f_at,f_bt
第1輪	1, 1, 2	s→a→t，2	2, 0, 0, 2, 0
第2輪	1, 1, 2	s→b→t，2	2, 2, 0, 2, 2
第3輪	1, 2, 3	s→a→b→t，1	3, 2, 1, 2, 3
終止檢查	∞, ∞, ∞	無路	值5，cut {s}容量5

Edmonds–Karp：殘餘邊數最短，不是容量最小 （續）



僅畫r>0的弧；綠色是本輪最少邊數路，δ依次為2、2

第3輪與終止：從殘餘圖讀出割



第3輪δ=1；最終s無出弧，流值5等於cut {s} 容量5

第3輪走s→a→b→t，瓶頸min(1,1,1)=1，流值4→5。更新後只剩a→s:3、b→s:2、b→a:1、t→a:2、t→b:3。從s只能到自己，所以S={s}；原圖跨割容量3+2=5，與流值相等。BFS的∞表示不可達，不是容量為∞。

BFS 距離不下降的反證

增廣前後記d、d'。假設有d'(v)<d(v)，選這類點中d'(v)最小者；令u為新最短路的前驅，d'(u)=d'(v)−1。u不是更小的下降反例，所以d(u)≤d'(u)。

若uv本來就在residual，則d(v)≤d(u)+1≤d'(v)，矛盾。因此uv是剛出現的反向邊，vu在剛才BFS增廣路上。舊最短路的前綴也最短，故d(u)=d(v)+1。

合併得d(v)=d(u)−1≤d'(u)−1=d'(v)−2，與d'(v)<d(v)矛盾。原不可達、新可達的∞情形也適用：上述有限前驅關係會迫使舊v可達。所以每點距離單調不減。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

臨界邊再次飽和之前，距離至少加二

每次增廣至少一條critical邊uv達到瓶頸δ，殘餘變0；此時d(v)=d(u)+1。它要重現，之後必沿vu反向增廣；該輪d'(u)=d'(v)+1。由距離不下降，d'(u)≥d(v)+1=d(u)+2。

因此同一方向每次重新成為critical，尾點距離至少增加2；仍在s-t簡單路上時距離≤n−2，所以每方向O(n)次。候選方向至多2m，總O(nm)次增廣。

每輪BFS／更新O(m+1)，m≥1時總$O(n+m+nm^2)$，通常省初始化寫$O(nm^2)$；m=0直接零流O(n)。有理容量共分母整數化後，所有值的bit長受原輸入長度多項式控制，輪數又與數值無關，因此也是多項式bit演算法。$O(nm^2)$先指算術操作界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逃生問題：限制邊，和限制格點是兩回事



只限制邊會算出2；限制每個格點的通過人數才得到1

座標採(row,column)。可走格點為{(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}，其他格點封閉。兩人從u=(3,2)、v=(2,3)出發，出口為邊界x=(3,5)、y=(5,3)。只限制每條邊容量1，兩條不同路可各送1，卻都經過z=(3,3)。**若題目禁止共用格點，這個流值2並不代表兩人都能逃出。**

把每個格點拆成「進入」與「離開」

對k×k網格，令q為可走格點數、p為不同起點的人數、b為可用邊界出口數。禁止共用任何格點時，建構如下：

原限制	網路表示
每格至多通過一人	每個v拆成v_in→v_out，容量1
可移到相鄰可走格	u_out→v_in與v_out→u_in，各容量1
每個起點有一人	source→起點_in，容量1
從可走邊界離開	邊界_out→sink，容量1

障礙格不建立頂點或邊。起點若已在邊界，也要經自己的內部容量邊再離開。網路有N=2q+2個頂點，邊數M≤q+4k(k−1)+p+b；完整無障礙網格的b=4k−4只適用k≥2，k=1時b=1。

為何最大流等於p，才能讓所有人逃出？

若有p條格點互不重複的逃生路，沿對應網路路徑各送1，所有容量與守恆都滿足。反過來，整數容量保證存在整數最大流；將流分解成source–sink路徑與循環，移除循環不改流值。值為p時，p條source出邊全部飽和；每條內部容量1使任何兩條路都不能使用同一格點。投回原網格便得到每個人的逃生路。這證明兩方向都成立。

本例所有出口路都經z。拆點後的z_in→z_out容量1構成一個割，故最多一人；路u→z→(3,4)→x又給可行流1，所以最大值恰為1。只做邊容量的模型有兩條邊不共用的路，最大值為2；**兩個答案不同，是因為限制不同。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.3 Maximum bipartite matching

二分圖L∪R的matching是不共端點的邊集合。建立s→每個左點容量1、原邊l→r容量1、每個右點→t容量1。

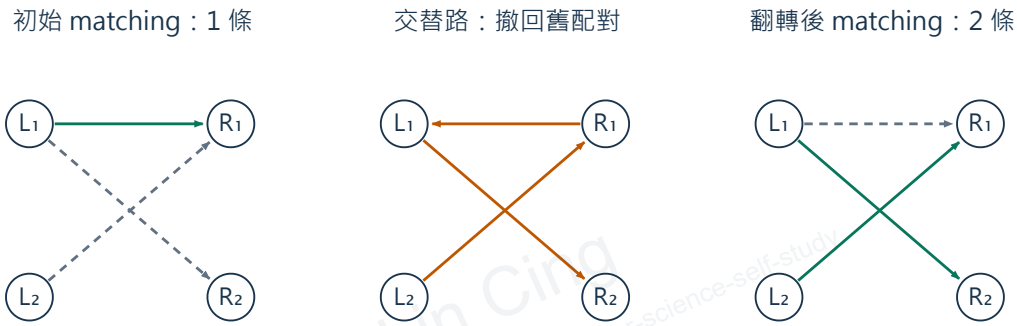
給matching M，沿每條s→l→r→t送1，端點互斥使容量不超，值|M|。反向給整數流：每個左點只有一條容量1入邊，守恆與整數性使其至多一條原匹配出邊為1；右點同理。因此f(l,r)=1的邊形成matching，大小等於流值。

整數增廣保證存在整數最大流，所以最大matching=最大flow。不能說每個最優流皆逐邊整數：$K_{2,2}$的四條左→右邊各送1/2、source與sink邊各1，也有最優值2，卻不能把四條正流邊全選作matching。

最大matching≤min(|L|,|R|)，每輪增1。新網路m'=|E|+|L|+|R|，直接Ford–Fulkerson成本O(|V|+m'+m'・min(|L|,|R|))。有孤點時不能把m'無條件寫成Θ(|E|)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條交替路實際翻轉哪些邊



$L_2 \to R_1 \to L_1 \to R_2$：未配、已配反向、未配

綠實線 = matching；灰虛線 = 未選原邊；橘箭頭 = residual 交替路

L_1可配R_1/R_2，L_2只可配R_1。先配L_1R_1後仍有residual路$s\to L_2\to R_1\to L_1\to R_2\to t$。反向$R_1\to L_1$撤回舊匹配，另取$L_2R_1$、$L_1R_2$，大小由1增2。

Alternating path的未配／已配邊交替；翻轉後每個內點仍接一條matching邊，兩個原未配端點各新增一條。

Kőnig 等式的可重建證明

取最大matching的residual，Z為從s可達點，$C=(L\setminus Z)\cup(R\cap Z)$。對原邊lr，若$l\in Z, r\notin Z$：未配邊有正向residual使r可達；已配時$s\to l$已飽和，能到l的最後邊只能從其匹配右點r反向而來，仍使r可達。兩種都矛盾，所以C覆蓋全部原邊。

每條matching邊的兩端同時可達或不可達：右端可沿反向到左；已配左端可達必先經它的右端。因此C對每條matching恰取一端。未配左點都可由s到達，不入C；未配右點不可達，否則能增廣，也不入C。所以$|C|=|M|$。

任一cover至少為matching每邊取一個不同端點，故二分圖最小cover大小=最大matching大小。一般圖不成立，例如三角形兩值為2與1。

Hall 條件的充分性

要匹配L全部點，必要條件是對每個A⊆L都有|N(A)|≥|A|，因matching給A不同鄰居。

若最大matching未覆蓋L，從未配左點做交替可達搜尋，得到左集合A、右集合B。前頁正反向論證給B=N(A)。所有B點已配，否則可增廣；它們的匹配左端都在A，而A還有至少一個起始未配左點。因此|A|>|B|=|N(A)|，違反Hall條件。

所以必要條件也充分。L空時空matching已覆蓋L，無須尋找未配起點。此結論講的是覆蓋全部左側；若左右同大，才同時成為兩側的perfect matching。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.4 Push-relabel algorithms

Preflow仍滿足反對稱與容量界，但允許暫存excess e(u)=Σ_v f(v,u)≥0，u≠s。Active只指u≠s,t且e(u)>0；sink雖累積excess，仍不discharge。

有效height滿足h(s)=n、h(t)=0，每條residual uv有h(u)≤h(v)+1。初始化其他height與f/e全0，設h(s)=n並飽和source出邊：f(s,v)=c(s,v)、f(v,s)=−c(s,v)，e(v)+=c(s,v)、e(s)−=c(s,v)。

初態所有非source excess非負；source無正殘餘出邊，其他邊從height0出發也滿足標號。Height不是每輪BFS的真距離，而是約束殘餘邊的有效標號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Push／Relabel 的完整條件

```
push(u,v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u],c(u,v)-f(u,v))
    f(u,v)+=d; f(v,u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u,v)>0)
```

Admissible指正殘餘、恰下降1層。Push後殘餘0稱saturating；否則nonsaturating，必使e(u)=0；兩者相等按saturating計。

Active必有正殘餘出邊，否則每個f(u,v)=c(u,v)≥0，導致e(u)=−Σf(u,v)≤0，矛盾。故relabel的min非空。無admissible時，整數有效標號使h(u)≤所有殘餘鄰居height，新height至少增1。

基本操作保持 Preflow 與有效 Height

Push不使e(u)變負，其他非source excess只增；容量／反對稱同增廣證明。Height不變，唯一可能新增的反向vu滿足h(v)=h(u)−1，所以仍有效，但不會admissible。

Relabel不改f/e。新h(u)=1+min h(v)使u出邊有效；對入邊wu，舊有效性有h(w)≤old_h(u)+1≤new_h(u)，所以入邊仍有效，且不admissible。Relabel後至少一條取到最小鄰居height的出邊成admissible。

Admissible network是DAG：沿每邊height減1，繞長k>0的cycle會推出h=h−k，矛盾。Push不新增admissible邊，relabel只可能新增自己的出邊，這兩點支撐後面的list拓撲不變量。

Active 點總有路回 Source，故 Height 有界

令R為active點u在residual可達集合，假設s∉R。R內非source excess皆非負、e(u)>0，所以 $\Sigma_{(v\in R)}e(v)>0$。但R沒有正殘餘出邊，故對v∈R,w∉R，f(v,w)=c(v,w)≥0；內部反對稱抵消給

$\Sigma_{(v\in R)}e(v)=-\Sigma_{(v\in R,w\notin R)}f(v,w)\le 0$，矛盾。

所以u有至多n−1邊的residual簡單路回s。沿路相加有效標號得h(u)≤h(s)+n−1=2n−1。每次relabel時u仍active，故新height同受此界；其餘時height不變。每內點最多2n−1次relabel，全圖$O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

終止時為最大流，還需另證一定終止

若全部內點excess0，preflow恢復守恆，是可行flow。若residual有s-t簡單路、長k≤n−1，沿有效標號得到n=h(s)≤h(t)+k=k≤n−1，矛盾。因此無增廣路，由max-flow min-cut得最大流。

Sink得到的excess是流值；一開始推出的多餘流可以回source，不必全進sink。

「終止時正確」不等於「必會終止」。接著分開限制relabel、saturating push與nonsaturating push數量，取得容量值無關的有限界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

飽和 Push 總數 O(nm)

固定方向uv。首次飽和時h(u)=h(v)+1且殘餘歸0；要再正向推送，必先有vu反向push恢復殘餘。該步h(v)=h(u)+1，height又不降，所以v至少比前次增2。

到uv再次飽和，u又需比v高1，因此u也至少比前次正向飽和時高2。Height≤2n−1，每個方向最多O(n)次；候選方向≤2m，總O(nm)。

此界依高度差與單調性，而非每次δ大小，對小數容量也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非飽和 Push 的勢能計費

取Φ=Σ_(active u)h(u)，初始0、始終非負。Relabel增加Φ恰為該height增加量，所有點全程height總增加≤(n−2)(2n−1)=O(n^2)。

飽和push只可能使目的點新active，Φ增加至多2n−1；共O(nm)次，正增加共O(n^2m)。非飽和push清空u；若v新active，最多加h(v)=h(u)−1，若v已active或為終端則不加。因此Φ至少減1。

由非負勢能，這類下降次數不超全部正增加，為O(n^2+n^2m)，m≥1時O(n^2m)。連同relabel與飽和push，generic基本操作共O(n^2m)；m=0時初始化後已無active，直接終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.5 The relabel-to-front algorithm

標準linked list L始終含V\{s,t}全部點，包括e=0者；不是只存active。固定N(u)={v：c(u,v)>0或c(v,u)>0}，含反向候選。current跨discharge保存：

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u,v)>0 and h[u]==h[v]+1:
            push(u,v) # 此步不前進current
        else: current[u]=next-neighbor(v)
```

飽和後下一迭代才略過該邊；若推送已清空u，立刻返回，current仍指原鄰居。不能把「每push自動前進」的另一版程式與此trace混搭。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

略過的 Current Arc 為何不用立即重看

固定u高度不變的一段。若uv因殘餘0略過，未來只能經vu反向push恢復；當時h(v)=h(u)+1，新uv往高處走，仍非admissible。若有殘餘但不admissible，有效標號給h(u)≤h(v)，v之後只升高，也不會成下降1層。

亦可從一般性質看：push不新增admissible邊；鄰居relabel後無admissible入邊。只有u自身relabel才可能新增自己的admissible出邊，屆時current重設。

所以掃到NIL表示所有已略過邊現在仍不可推，relabel確實合法；不是因為「曾經看過」就忽略邊的狀態改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Move-front 後讀取新位置的 Next

```
initialize_preflow()
L=all vertices except s,t, in any fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # 移動後的新位置
```

Move在整次discharge結束後做，不是每次內部relabel就做。中間u移到head後，next是舊head，會重查先前可能收到excess的點。不能保存舊next直接跳過它們。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

List 不變量的兩種分支

每次外圈測試：(1)L為非s/t點的admissible network拓撲序；(2)u之前的點皆無excess。初態非source高度皆0，彼此無admissible邊，任意L符合，u在head。

若discharge無relabel，push不新增admissible邊，L仍拓撲有序；只把excess推到後方點或終端，不影響前綴。u最後清空，下一個u以前仍全無excess。

若曾relabel，最後一次relabel後u無admissible入邊，其後push也不新增；移到head保持拓撲序。新next之前只有剛清空的u，前綴條件恢復，其他點相對順序不變。

到u=NIL時，全部L皆為無excess前綴，故守恆，結合generic正確性即最大流。這個證明不能直接套到另一種只存active點的排程。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五點網路的初始化與 a 的完整 Discharge

容量s→a=4、s→b=3、s→c=2、a→b=2、a→t=1、b→c=2、b→t=4、c→t=5。固定N(a)=[b,t,s]、N(b)=[c,t,a,s]、N(c)=[t,b,s]，L=[a,b,c]。

初始化h(s)=5、其餘0，飽和source出邊。以下在每個push/relabel剛完成時記current，未合併下一次掃描：

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)
初始化	4, 3, 2, 0	0, 0, 0	b,c,t
a掃b,t,s到NIL；relabel	4, 3, 2, 0	1, 0, 0	b,c,t
a→b推2	2, 5, 2, 0	1, 0, 0	b,c,t
略b到t；a→t推1	1, 5, 2, 1	1, 0, 0	t,c,t
掃t,s到NIL；再relabel	1, 5, 2, 1	6, 0, 0	b,c,t
略b,t到s；a→s退1	0, 5, 2, 1	6, 0, 0	s,c,t

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五點網路的初始化與 a 的完整 Discharge （續）

a的height從0變6，結束後move-front；因原在head，L不變，接著掃b。

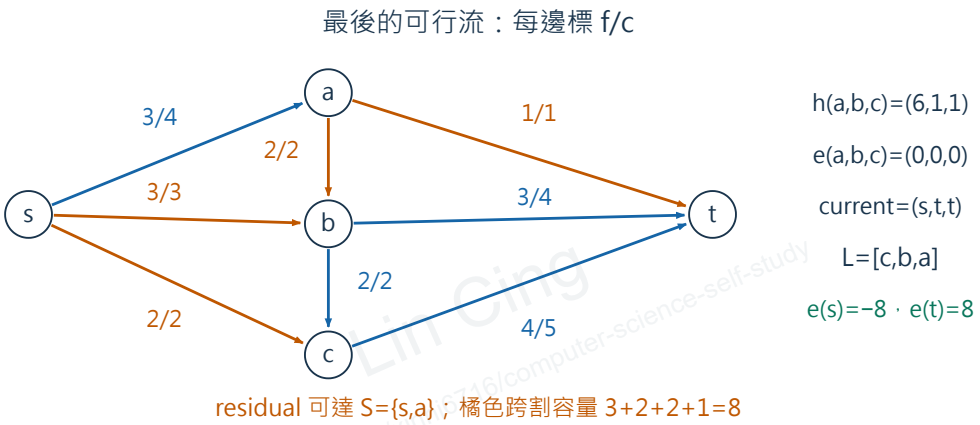
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

b、c 的 Discharge 與真正移動時點

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)	L
b掃c,t,a,s到NIL；relabel	0, 5, 2, 1	6, 1, 0	s,c,t	a,b,c
b→c推2	0, 3, 4, 1	6, 1, 0	s,c,t	a,b,c
略c到t；b→t推3	0, 0, 4, 4	6, 1, 0	s,t,t	a,b,c
b結束後移front	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
再掃a，e=0無動作	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
c掃t,b,s到NIL；relabel	0, 0, 4, 4	6, 1, 1	s,t,t	b,a,c
c→t推4；結束後移front	0, 0, 0, 8	6, 1, 1	s,t,t	c,b,a

下一點是新位置的b，接著a，皆無excess，最後NIL。b→c後t仍1，只有b→t後才變4。最終source淨流3+3+2=8、sink淨入1+3+4=8。

以 Cut 獨立核對最後的最大流



a收3送2+1；b收3+2送2+3；c收2+2送4。殘餘可達S={s,a}，跨割原容量3+2+2+1=8，等於可行流值8。初始推出9，其中1由a退回source，所以最後淨流值為8；這不是漏流。割證書獨立於height和排程。

$O(n^3)$ 的完整計費

總relabel $O(n^2)$。兩次相鄰relabel間稱一個phase；沒有relabel時discharge使u沿L向後走，最多n次，含relabel的一次使進入新phase。因此discharge呼叫$O(n^3)$；move-front持有效雙向串列handle可$O(1)$。

每點current在兩次自身relabel之間掃鄰居一次，自身relabel $O(n)$次，全部advances與重標找min合計$O(n\Sigma deg(u))=O(nm)$。飽和push $O(nm)$。每次discharge中非飽和push一旦發生就清空u並返回，所以至多一次，總$O(n^3)$。

合計$O(n^3+nm)=O(n^3)$，因簡單容量網路$m \le n(n-1)$。稀疏鄰接／反向記錄、height/excess/current與L共$O(n+m)$空間。這先是算術次數界，容量加法與比較仍依實際bit長付費。

Generic 操作次數不是免費的選步成本

若每步都掃全部頂點找active，再重掃全部鄰居，不能僅由基本操作$O(n^2m)$就宣稱同一時間界。可另維護active集合：push只改兩端excess，以handle與標記$O(1)$加入／移除，任取active點處理。

鄰居用同一current規則，全部advances與relabel掃描$O(nm)$，push本身$O(1)$，才能實現初始化後$O(n^2m)$算術時間。Relabel-to-front以固定全體串列及discharge另得$O(n^3)$，不是把active集合改個名字。

整數Ford–Fulkerson依F^*、Edmonds–Karp依nm^2、generic依n^2m、relabel-to-front依n^3，是不同機制的最壞操作界，不直接代表某台機器的實測速度。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 26。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §26.5.

《演算法_wjungle》手寫筆記，網路流、逃生問題與節點容量，PDF實體第37–42頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study